

Lab no 7

Date: 03/03/2026

RD PARSER FOR DECLARATION STATEMENTS

Name: Satyam Singh

Reg no: 230905256

Roll No: 37

CSE B2

Lab exercise:

Sample C grammar:

Data Types	:	int, char
Arrays	:	1-dimensional
Expressions	:	Arithmetic and Relational
Looping statements	:	for, while
Decision statements	:	if, if – else

Program $\rightarrow$ main () { declarations statement-list }

Declarations $\rightarrow$ data-type identifier-list; declarations | ϵ

data-type $\rightarrow$ int | char

identifier-list $\rightarrow$ id | id, identifier-list | id[number] , identifier-list | id[number]

statement_list $\rightarrow$ statement statement_list | ϵ

statement $\rightarrow$ assign-stat; | decision_stat

assign_stat $\rightarrow$ id = expn

expn $\rightarrow$ simple-expn eprime

epime $\rightarrow$ relop simple-expn | ϵ

simple-exp $\rightarrow$ term seprime

seprime $\rightarrow$ addop term seprime | ϵ

term $\rightarrow$ factor tprime

tprime $\rightarrow$ mulop factor tprime | ϵ

factor $\rightarrow$ id | num

decision-stat $\rightarrow$ if (expn) {statement_list} dprime

dprime $\rightarrow$ else {statement_list} | ϵ

relop $\rightarrow$ == | != | <= | >= | > | <

addop $\rightarrow$ + | -

mulop $\rightarrow$ * | / | %

Grammar 7.1

1. For given subset of grammar 7.1, design RD parser with appropriate error messages with expected character and row and column number.

```
Program → main () { declarations assign_stat }  
declarations → data-type identifier-list; declarations | ε  
data-type → int | char  
identifier-list → id | id, identifier-list  
assign_stat → id=id; | id = num;
```

Ans)

```
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <ctype.h>  
  
#include <string.h>  
  
#define MAIN 1  
  
#define INT 2  
  
#define CHAR 3  
  
#define ID 4  
  
#define NUM 5  
  
#define ASSIGN 6  
  
#define SEMI 7  
  
#define COMMA 8  
  
#define LPAREN 9  
  
#define RPAREN 10  
  
#define LBRACE 11
```

```
#define RBRACE 12
```

```
#define END 13
```

```
typedef struct {
```

```
    int type;
```

```
    char lexeme[50];
```

```
    int row;
```

```
    int col;
```

```
} Token;
```

```
Token token;
```

```
FILE *fp;
```

```
int row = 1, col = 0;
```

```
Token getNextToken();
```

```
void Program();
```

```
void declarations();
```

```
void data_type();
```

```
void identifier_list();
```

```
void assign_stat();
```

```
void match(int expected, char *expectedName);
```

```
void error(char *expected);
```

```
void error(char *expected)
```

```
{  
    printf("\nSyntax Error!\n");  
    printf("Row: %d Column: %d\n", token.row, token.col);  
    printf("Expected: %s\n", expected);  
    printf("Found: %s\n", token.lexeme);  
    exit(1);  
}
```

```
void match(int expected, char *expectedName)
```

```
{  
    if(token.type == expected)  
        token = getNextToken();  
    else  
        error(expectedName);  
}
```

```
Token getNextToken()
```

```
{  
    Token t;  
    char ch;  
    int i = 0;  
  
    ch = fgetc(fp);
```

```
while(ch == ' ' || ch == '\t' || ch == '\n')
{
    if(ch == '\n')
    {
        row++;
        col = 0;
    }
    ch = fgetc(fp);
}
t.row = row;
t.col = col;

if(ch == EOF)
{
    t.type = END;
    strcpy(t.lexeme, "EOF");
    return t;
}
col++;
if(isalpha(ch))
{
```

```
t.lexeme[i++] = ch;
ch = fgetc(fp);
col++;

while(isalnum(ch))
{
    t.lexeme[i++] = ch;
    ch = fgetc(fp);
    col++;
}
t.lexeme[i] = '\0';
fseek(fp, -1, SEEK_CUR);
col--;
if(strcmp(t.lexeme, "main") == 0)
    t.type = MAIN;
else if(strcmp(t.lexeme, "int") == 0)
    t.type = INT;
else if(strcmp(t.lexeme, "char") == 0)
    t.type = CHAR;
else
    t.type = ID;
```

```
        return t;
    }
    if(isdigit(ch))
    {
        t.lexeme[i++] = ch;
        ch = fgetc(fp);
        col++;
        while(isdigit(ch))
        {
            t.lexeme[i++] = ch;
            ch = fgetc(fp);
            col++;
        }
        t.lexeme[i] = '\0';
        fseek(fp, -1, SEEK_CUR);
        col--;
        t.type = NUM;
        return t;
    }
    switch(ch)
```

```
{
    case '=': t.type = ASSIGN; strcpy(t.lexeme, "="); break;
    case ';': t.type = SEMI; strcpy(t.lexeme, ";"); break;
    case ',': t.type = COMMA; strcpy(t.lexeme, ","); break;
    case '(': t.type = LPAREN; strcpy(t.lexeme, "("); break;
    case ')': t.type = RPAREN; strcpy(t.lexeme, ")"); break;
    case '{': t.type = LBRACE; strcpy(t.lexeme, "{"); break;
    case '}': t.type = RBRACE; strcpy(t.lexeme, "}"); break;
    default:
        printf("Lexical Error at Row %d Column %d\n", row,
col);
        exit(1);
}
return t;
}

void Program()
{
    match(MAIN, "main");
    match(LPAREN, "(");
    match(RPAREN, ")");
    match(LBRACE, "{");
```



```
    declarations();
    assign_stat();
    match(RBRACE, "}");
}

void declarations()
{
    if(token.type == INT || token.type == CHAR)
    {
        data_type();
        identifier_list();
        match(SEMI, ";");
        declarations();
    }
}

void data_type()
{
    if(token.type == INT)
        match(INT, "int");
    else if(token.type == CHAR)
        match(CHAR, "char");
    else
```

```
        error("int or char");
    }
void identifier_list()
{
    match(ID, "identifier");

    if(token.type == COMMA)
    {
        match(COMMA, ",");
        identifier_list();
    }
}
void assign_stat()
{
    match(ID, "identifier");
    match(ASSIGN, "=");

    if(token.type == ID)
        match(ID, "identifier");
    else if(token.type == NUM)
        match(NUM, "number");
```

```
    else
        error("identifier or number");

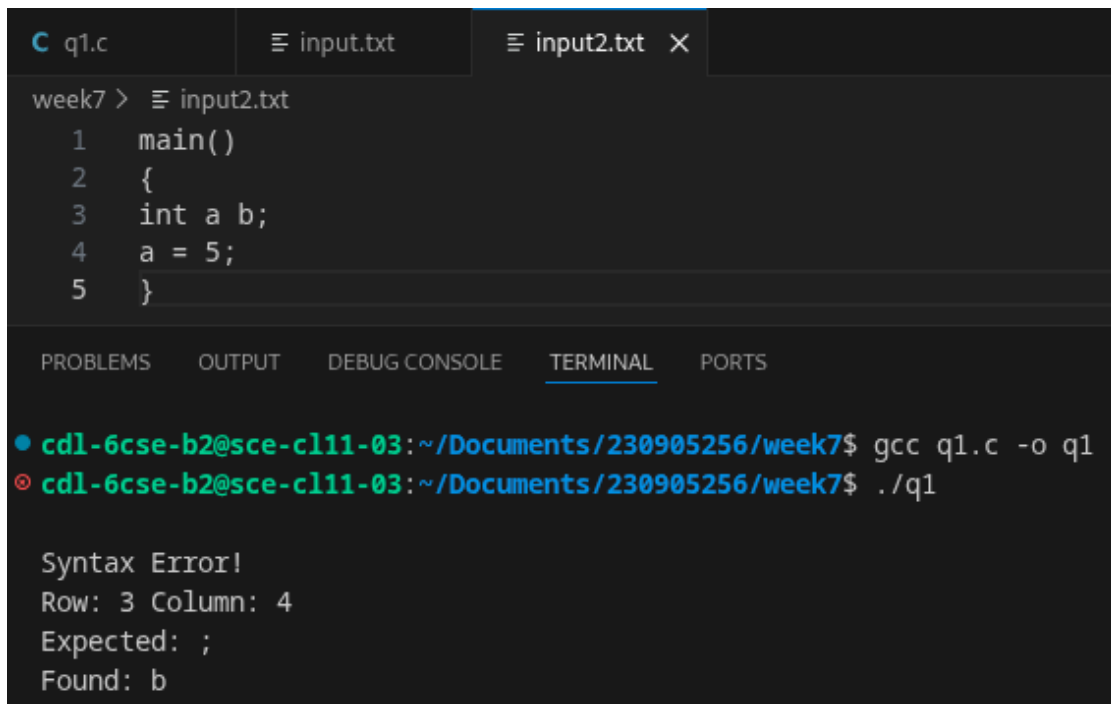
    match(SEMI, ";");
}

int main()
{
    fp = fopen("input2.txt", "r");
    if(fp == NULL)
    {
        printf("Cannot open file.\n");
        return 0;
    }
    token = getNextToken();
    Program();

    if(token.type == END)
        printf("\nParsing Successful!\n");
    else
        error("EOF");
```

```
fclose(fp);  
  
return 0;  
  
}
```

Output with error in input:



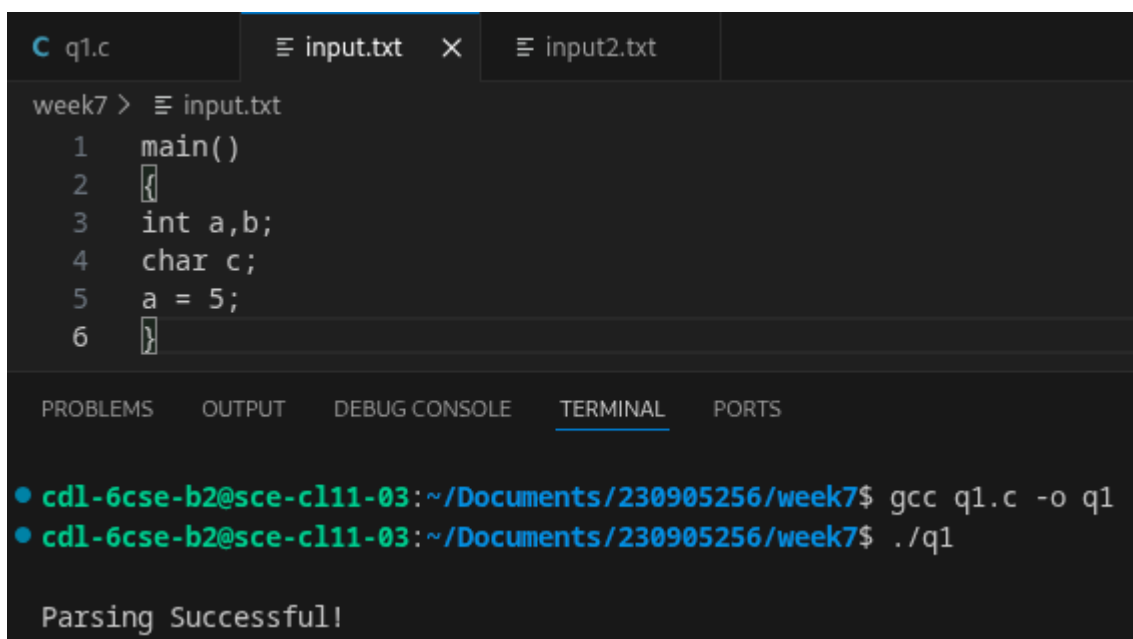
The screenshot shows a code editor with two tabs: 'q1.c' and 'input2.txt'. The code in 'q1.c' is as follows:

```
1 main()  
2 {  
3 int a b;  
4 a = 5;  
5 }
```

Below the code editor, the 'TERMINAL' tab is active, showing the following commands and output:

```
cdl-6cse-b2@sce-cl11-03:~/Documents/230905256/week7$ gcc q1.c -o q1  
cdl-6cse-b2@sce-cl11-03:~/Documents/230905256/week7$ ./q1  
  
Syntax Error!  
Row: 3 Column: 4  
Expected: ;  
Found: b
```

Output without Error:



The screenshot shows a code editor with two tabs: 'q1.c' and 'input2.txt'. The code in 'q1.c' is as follows:

```
1 main()  
2 {  
3 int a,b;  
4 char c;  
5 a = 5;  
6 }
```

Below the code editor, the 'TERMINAL' tab is active, showing the following commands and output:

```
cdl-6cse-b2@sce-cl11-03:~/Documents/230905256/week7$ gcc q1.c -o q1  
cdl-6cse-b2@sce-cl11-03:~/Documents/230905256/week7$ ./q1  
  
Parsing Successful!
```